ADDIS ABABA UNIVERSITY

College of Natural and Computational Sciences

Department of Computer Science

Task Assignment and Tracking for Workplace Teams at Mersa Media Institute (MMI)

Software Design Specification

PREPARED BY Group 17: -

1. Shalom Arbsie - UGR/9515/16
2. Tsegab Faskal - UGR/4455/16
3. Nahom Solomon - UGR/3155/16
4. Nolawi Assefa – UGR/7847/16
5. Nebiyu Amare – UGR/0791/16
6. Yonathan Markos – UGR/6562/16

ADVISOR: Aderaw Semma

Date: Saturday, November 29th

Table of Contents

1. Introduction

1.1 Purpose

The purpose of this System Design Document (SDD) is to translate the previously defined business requirements and workflow needs of Mersa Media Institute (MMI) into a complete technical design. This document describes how the system will be structured, how its components will interact, and how the design will support the development of the task management and communication platform. It serves as a blueprint for developers, guiding the implementation, integration, and testing phases.

1.1 General Overview

This system is a **web-based platform** that integrates task management, real-time communication, and an analytics dashboard into one environment. Its goal is to eliminate the need for multiple separate tools and provide a unified workflow for MMI's team.

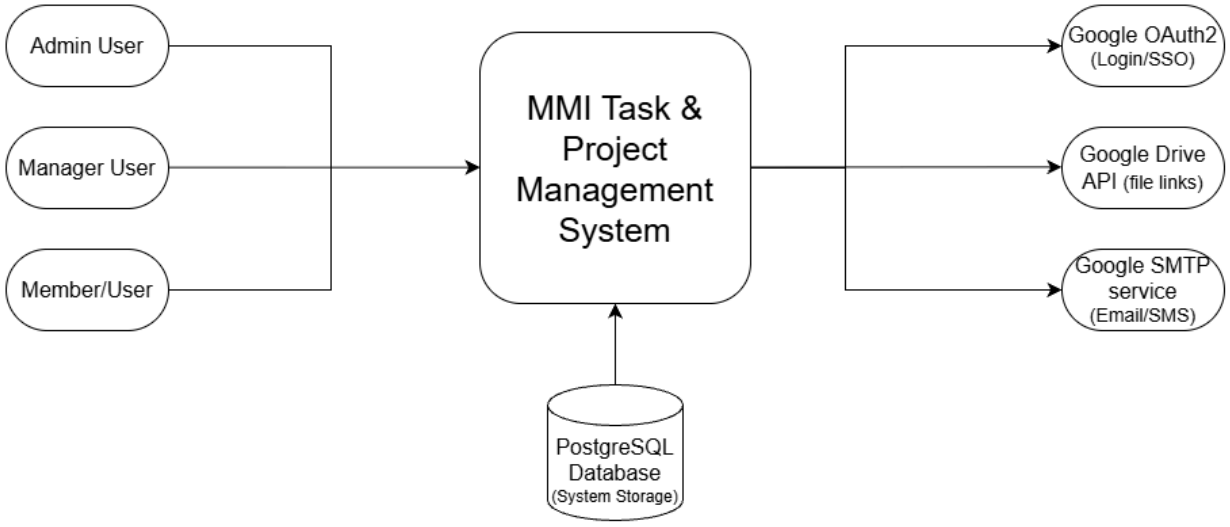
The basic design follows a **client–server architecture:**

- The **client** (frontend) handles user interaction, displaying tasks, chats, and analytics.
- The **server** (backend) manages data storage, task logic, messaging, and analytics aggregation.
- A **database layer** stores tasks, messages, user information, and activity data.

Design goals

- Provide a simple, intuitive interface for non-technical users.
- Ensure real-time communication within tasks.
- Maintain consistent and reliable data access.
- Enable easy scalability for future expansion.

High-Level system context



The system interacts with:

- **Users** (creators, editors, managers)
- **Web browsers** (client interface)
- **Database** (persistent storage)
- **Etc.**

Changes from earlier concept:

- The analytics component was emphasized more clearly than in early drafts to support MMI's performance monitoring needs.
- Communication features were integrated directly into tasks instead of through a separate messaging module.

1.3 Development Methods & Contingencies

The project uses a **modular, object-oriented design approach**.

Key characteristics of the approach:

- System functions are separated into modules (tasks, chat, analytics, user management).
- Components are designed as independent units that interact through well-defined interfaces.
- The system follows a **layered architecture** (presentation layer, application logic layer, data layer).

This approach was chosen because:

- It supports future scalability and extensions.

- It simplifies maintenance and debugging.
- It allows different team members to work on different modules independently.

Other methods considered:

- **Structured design** was considered but not used because the system requires components that can evolve independently.
- **Prototyping approach** is partially used for UI/UX since the system requires continuous feedback from MMI. Early mockups and layouts help validate usability before full implementation.

Contingencies

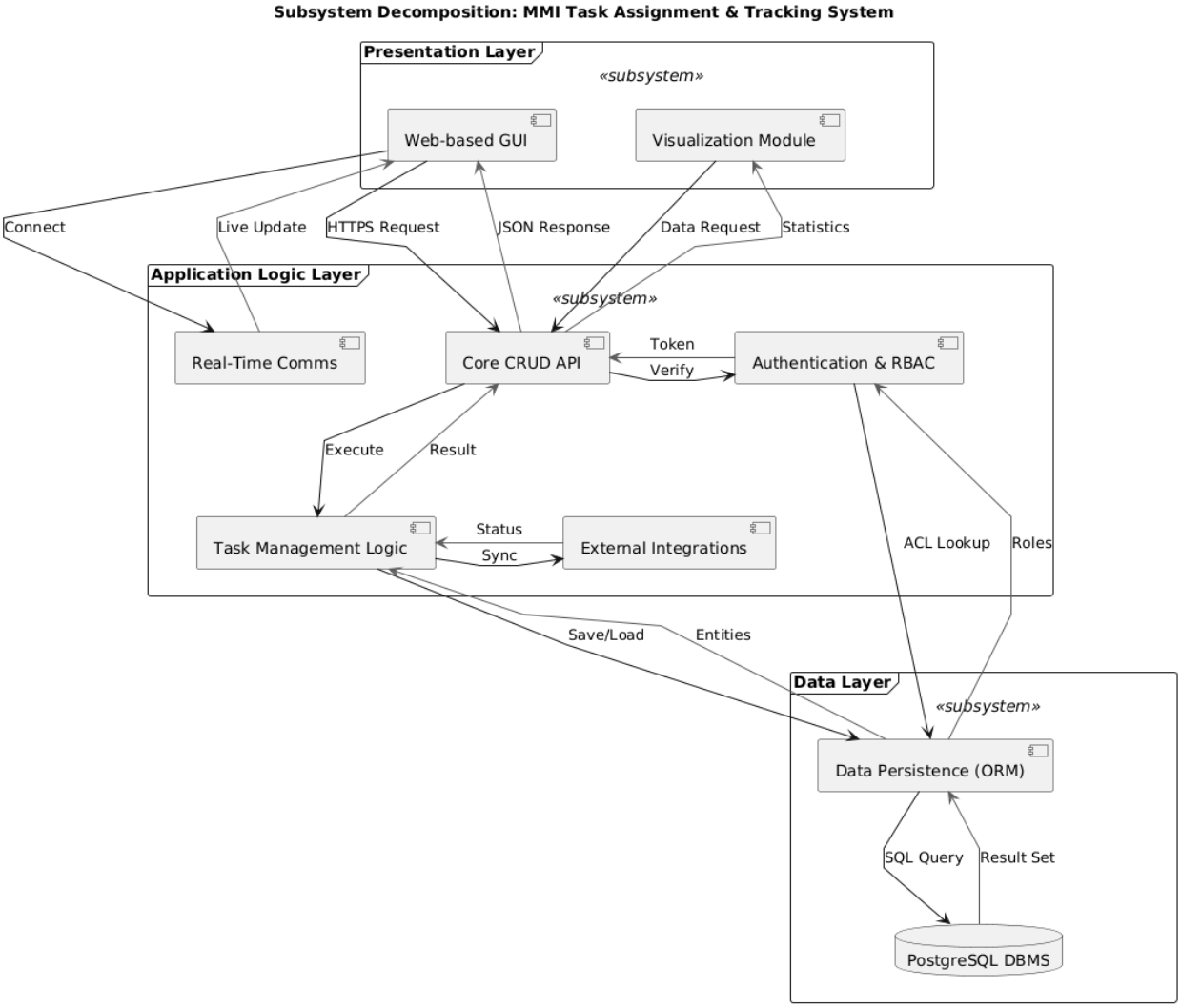
Potential issues that may impact system design:

- **Unclear hosting environment**: If MMI's hosting capabilities change, deployment architecture may need adjustments.
    - *Workaround*: Keep deployment-independent design so the system can run on any standard web server.
- **Real-time communication constraints**: If network conditions are unreliable for some users, real-time features may experience delays.
    - *Workaround*: Implement fallback asynchronous messaging if live updates fail.
- **User adoption changes**: Requirements may shift based on feedback from MMI staff during evaluation.
    - *Workaround*: Keep UI flexible and modular so features can be adjusted without redesigning the whole system.
- **Scalability concerns**: If MMI grows or other organizations adopt the tool, higher load might require optimization.
    - *Workaround*: Design system components to scale horizontally (more servers/modules added easily).

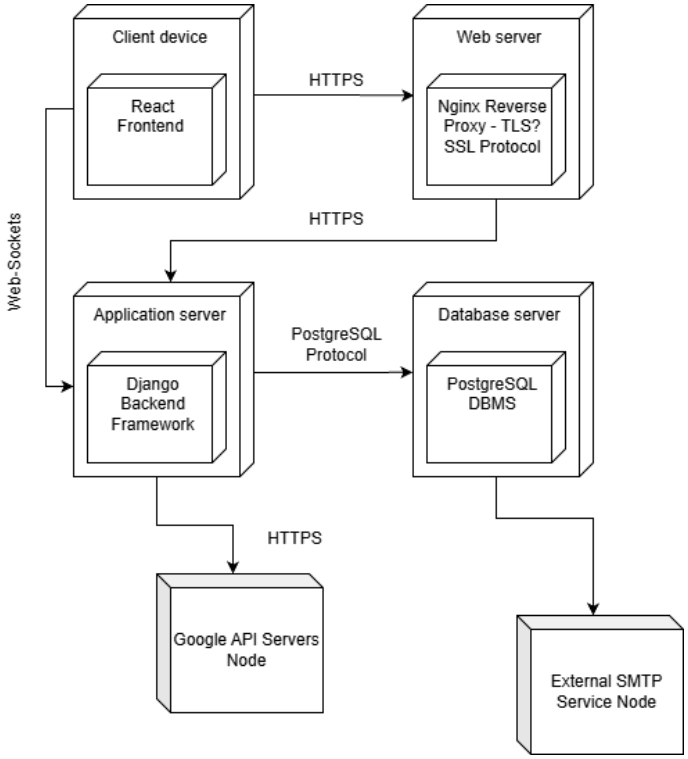
These contingencies allow the design to remain stable even if certain external factors change during development or deployment.

2. System Architecture

2.1 Subsystem decomposition



2.2 Hardware/software mapping



3. Object Model

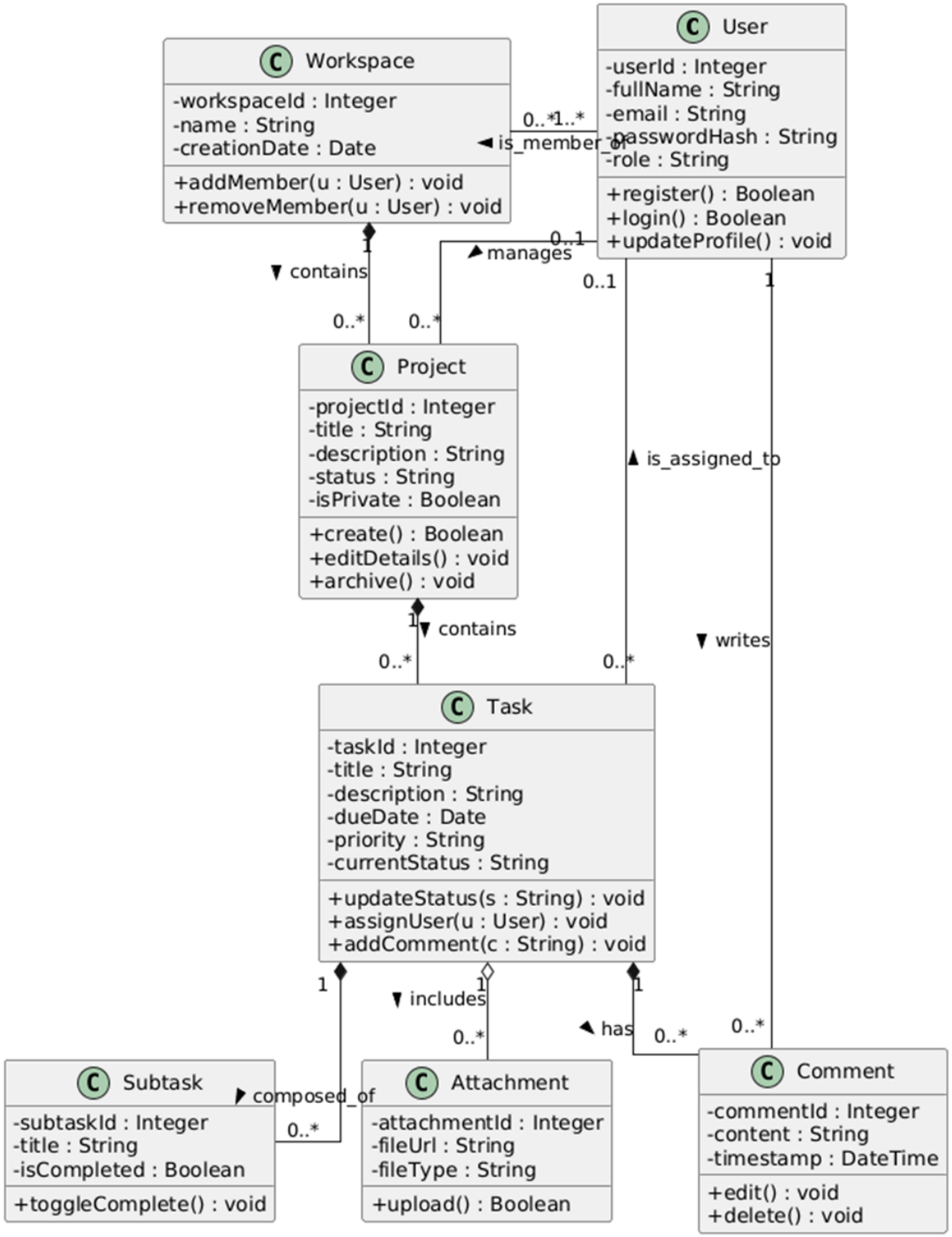
3.1 Class Diagram

The Class Diagram illustrates the static structure of the MMI Task Management System. It enforces the principle of encapsulation, where all class attributes are defined as private (-) to ensure data integrity, while methods are exposed as public (+) to allow interaction between objects.

Relationship Multiplicities:

- **Workspace (1) — Project (0..*):** This represents a "One-to-Many" composition. A single Workspace acts as a container that creates and holds zero or more Projects. If the Workspace is deleted, the Projects within it are also removed.
- **Project (1) — Task (0..*):** Similarly, a Project serves as the parent entity for Tasks. A Project must exist for a Task to be created.
- **User (0..1) — Task (0..*):** This association represents task assignment. A task may exist in the backlog without an assignee (0), or it may be assigned to a single specific user (1). Conversely, a single User may have multiple tasks assigned to them (*).
- **Task (1) — Subtask (0..*):** This is a composition relationship where Subtasks are dependent parts of a main Task.

*To preserve the visual integrity of the model and ensure that all private attributes, public operations, and strict multiplicity constraints remain clearly legible, the full **Class Diagram** is presented on the following page.*

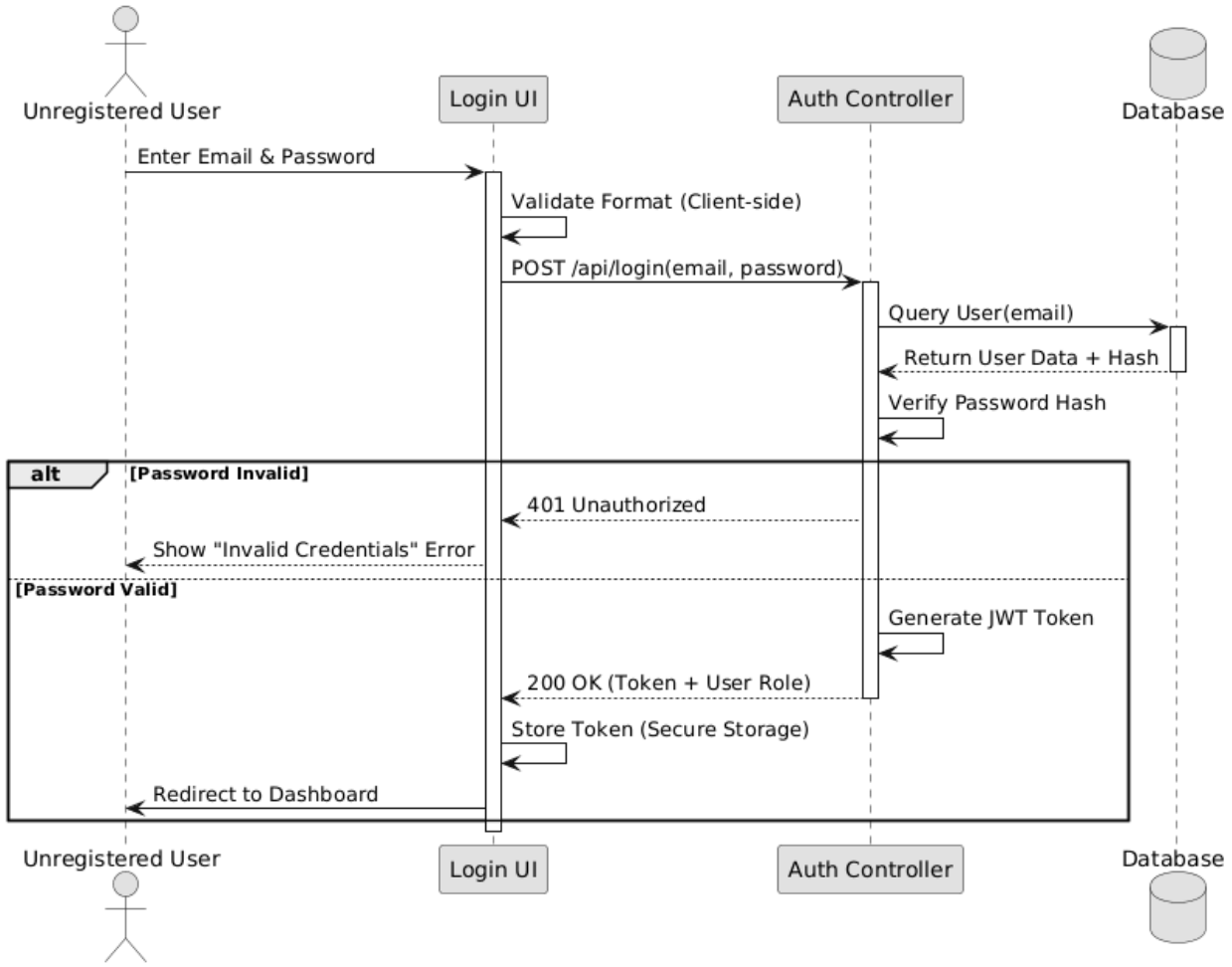


3.2 Sequence Diagram

The following sequence diagrams illustrate the interaction between actors and system objects for the primary Use Cases defined in the requirements.

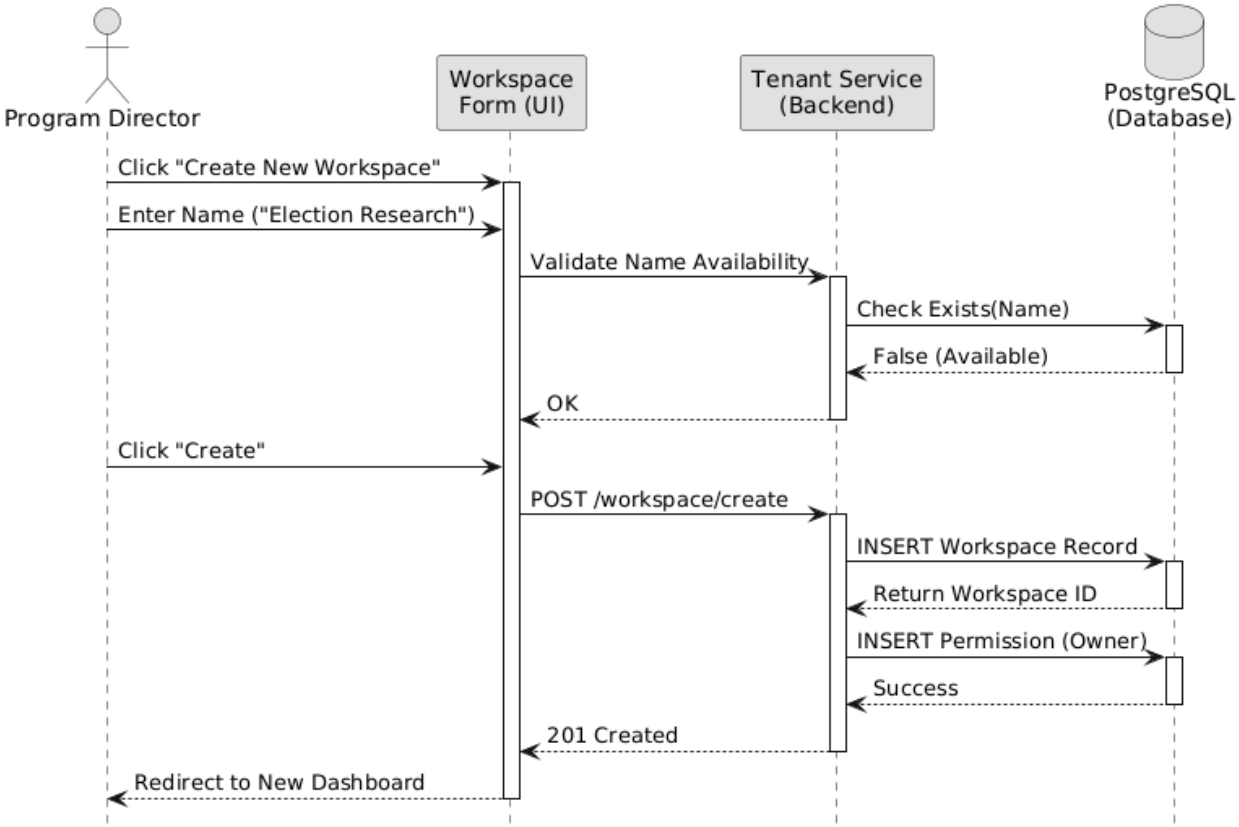
Scenario: User Authentication (Login)

This diagram details the process defined in SRS Section 3.2.1, ensuring users are authenticated and issued a secure session token (JWT) before accessing the workspace



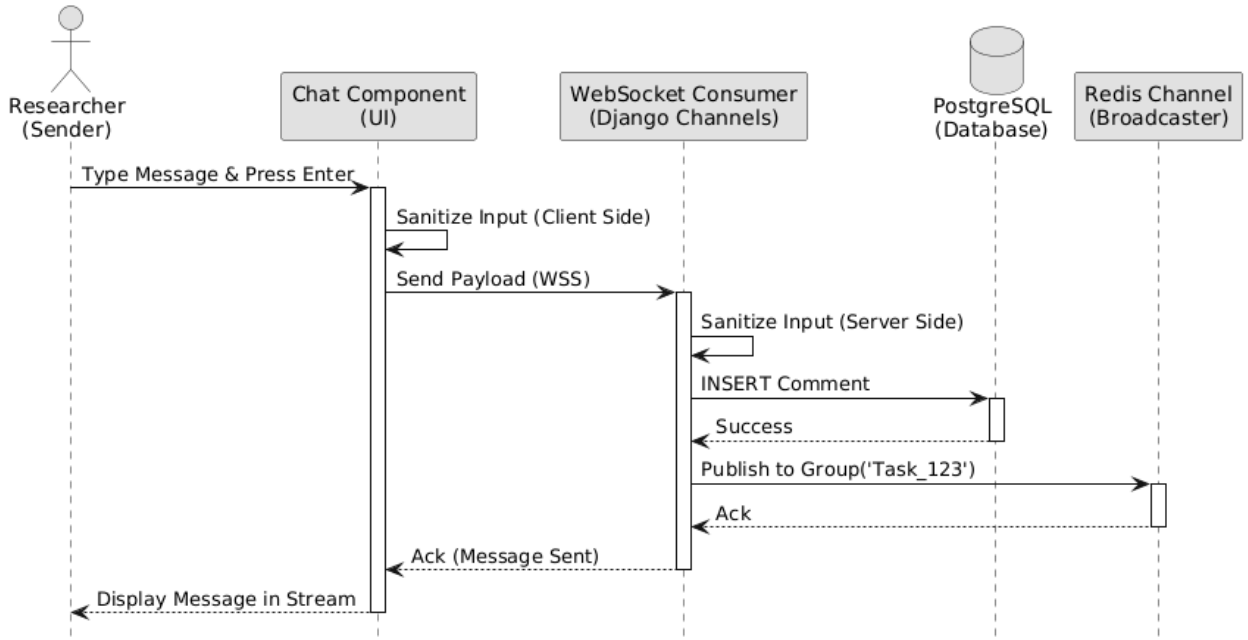
Scenario: Create Workspace

This diagram illustrates the "Program Director" initializing a new organizational unit. It details the validation logic where the system checks the Database for name uniqueness before committing the new record.



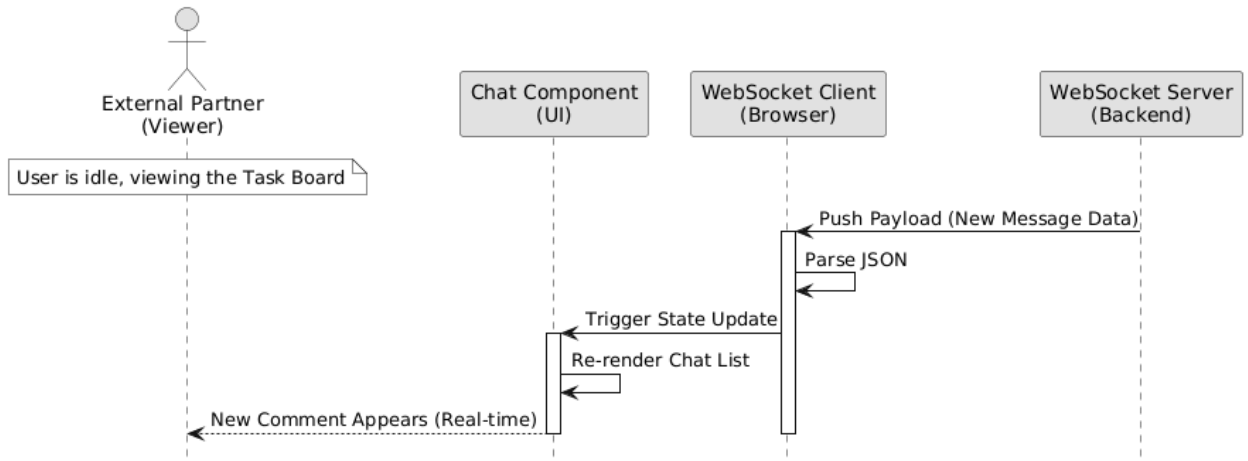
Scenario: Post Real-Time Comment

This diagram focuses on the **Sender's** interaction. It tracks the flow from the moment the Researcher types a message until the server acknowledges receipt and broadcasts the payload.
Note: This view terminates at the server broadcast to isolate the sender's context.



Scenario: Receive Real-Time Update

This diagram models the **passive reception** of data. It demonstrates how the **External Partner (Viewer)** receives a push update via WebSockets without initiating any action, causing the UI to re-render automatically.

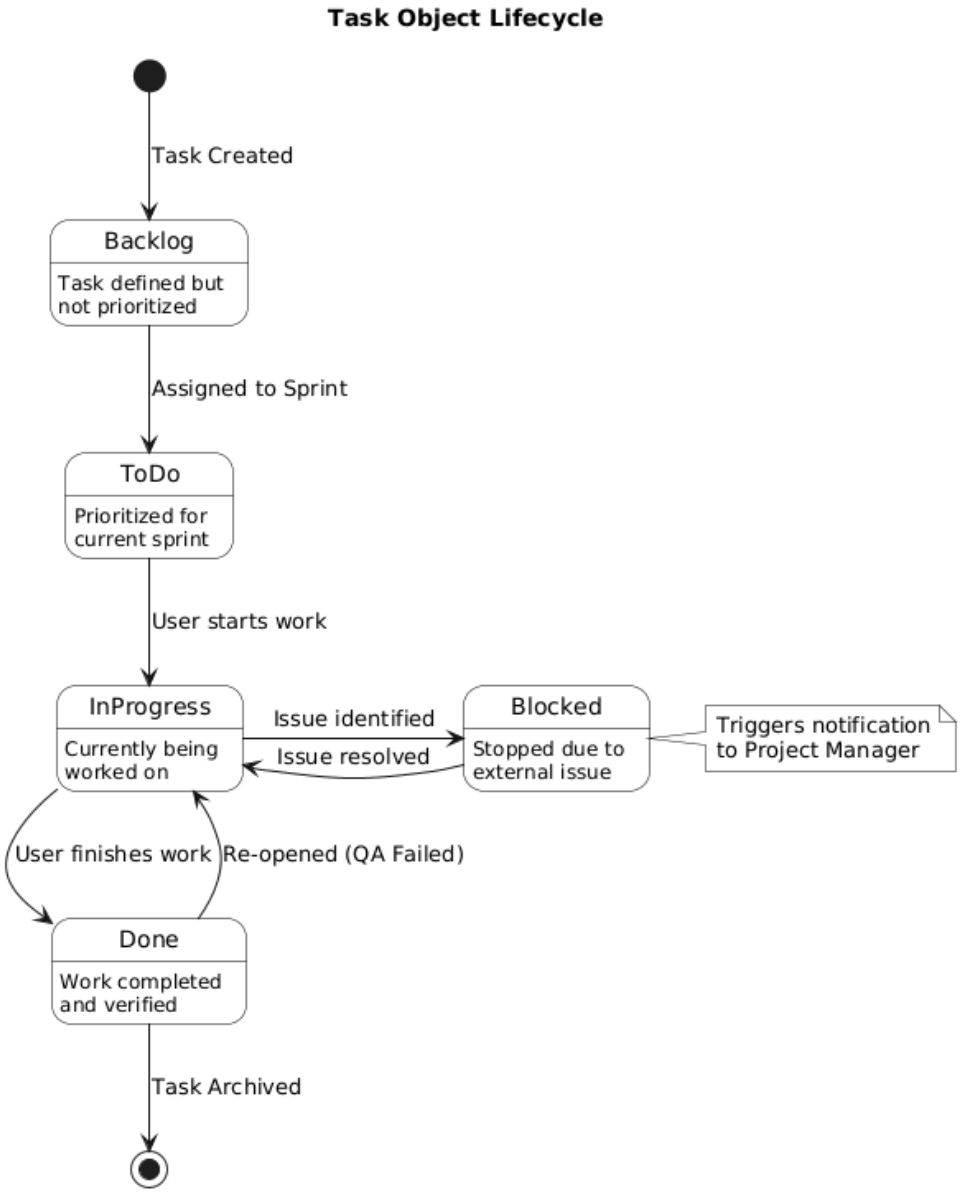


3.3 State chart Diagram (optional element)

State chart diagrams describe the dynamic behavior of an individual object as it moves through various states during its lifetime. For the MMI System, we have modeled the lifecycles of the two most critical entities: **Tasks** and **Projects**.

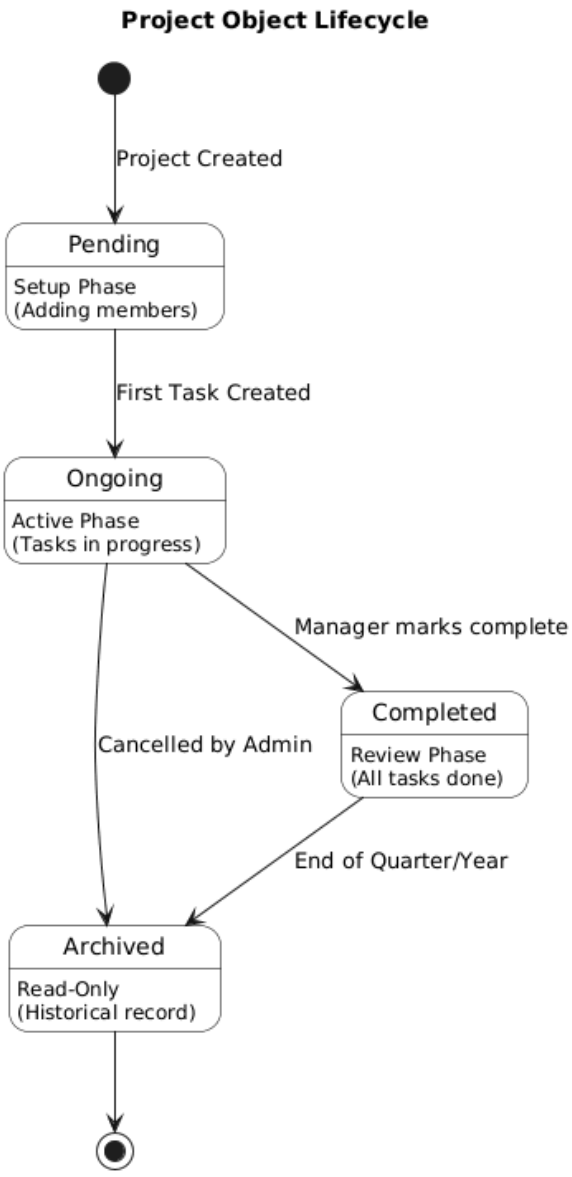
Task Lifecycle

The Task is the central unit of work in the system. Its state changes based on user actions (starting work, completing work) or blockers (dependencies). This diagram enforces the status workflow defined in SRS Section 3.2.3.



Project Lifecycle

While tasks represent daily work, Projects represent the broader goals of MMI. This state chart defines the high-level status of a project container, ensuring that projects are properly initialized before work begins and archived when finished.



4. Detailed Design

Classes below are the core domain objects for the Task Assignment & Tracking system with Slack and Google Calendar integrations. Each class includes a UML-style summary, attribute constraints (invariants), and operation pre/post-conditions following your template.

4.1 Workspace

Table: Workspace class

```
Workspace
+Id:UUID
+Name:String
+OwnerId:UUID
+createdAt:DateTime
+updatedAt:DateTime
+addMember()
+removeMember()
```

Table: Attributes description for Workspace class

Attribute	Type	Visibility	Invariant
Id	UUID	Public	Id <> NULL, immutable
Name	String	Public	Name <> NULL, length 3..80, no control chars
OwnerId	UUID	Public	Must reference existing User with role=Owner
createdAt	DateTime	Public	Auto-set on insert
updatedAt	DateTime	Public	Auto-updated on change

Table: Operation description for Workspace class

Operation	Visibility	Return type	Argument	Pre-Condition	Post Condition
addMember	Public	void	userId:UUID, role:Role	User exists; caller is Owner/Admin	Membership created or role updated; audit log appendedAdds entry to the

Operation	Visibility	Return type	Argument	Pre-Condition	Post Condition
					WorkplaceMember Table
removeMember	Public	void	userId:UUID	Caller is Owner/Admin; user is member	Membership removed; reassignment policy executed. Removes corresponding entry from the WorkplaceMember Table

4.2 User

Table: User class

```
User
+Id:UUID
+FullName:String
+Email:String
+Role:Enum
+Status:Enum
+createdAt:DateTime
+updateProfile()
```

Table: Attributes description for User class

Attribute	Type	Visibility	Invariant
Id	UUID	Public	Id <> NULL, immutable
FullName	String	Public	<> NULL, length 2..120
Email	String	Public	Unique; lowercase stored

Attribute	Type	Visibility	Invariant
Role	Enum(Owner,Admin,Manager,Member,Viewer)	Public	Not NULL
Status	Enum(Active,Disabled)	Public	Not NULL
createdAt	DateTime	Public	Auto-set

Table: Operation description for User class

Operation	Visibility	Return type	Argument	Pre-Condition	Post Condition
updateProfile	Public	void	profilePatch:JSON	Authenticated; fields allowed	Profile saved; audit entry recorded

4.3 Project

Table: Project class

```
Project
+Id:UUID
+WorkspaceId:UUID
+Name:String
+Description:String
+createdAt:DateTime
+updatedAt:DateTime
+addMember()
```

Table: Attributes description for Project class

Attribute	Type	Visibility	Invariant
Id	UUID	Public	Not NULL
WorkspaceId	UUID	Public	Must reference existing Workspace
Name	String	Public	Not NULL, length 3..100
Description	String	Public	≤ 1,000 chars
createdAt	DateTime	Public	Auto-set

Attribute	Type	Visibility	Invariant
updatedAt	DateTime	Public	Auto-updated

Table: Operation description for Project class

Operation	Visibility	Return type	Argument	Pre-Condition	Post Condition
addMember	Public	void	userId:UUID	User in same workspace	Project membership added

4) Task

Table: Task class

```
Task
+Id:UUID
+ProjectId:UUID
+Title:String
+Description:String
+AssigneeId:UUID
+Status:Enum
+Priority:Enum
+DueAt:DateTime
+GcalEventId:String
+createdAt:DateTime
+updatedAt:DateTime
+createSubtask()
+changeStatus()
+assignTo()
+linkToCalendar()
```

Table: Attributes description for Task class

Attribute	Type	Visibility	Invariant
Id	UUID	Public	Not NULL
ProjectId	UUID	Public	Must reference existing Project
Title	String	Public	Not NULL, length 1..140

Attribute	Type	Visibility	Invariant
Description	String	Public	≤ 10,000 chars
AssigneeId	UUID	Public	Optional; if set, user must be in project workspace
Status	Enum(Backlog, ToDo,InProgress,Blocked,Done, Cancelled)	Public	Not NULL; transitions must follow workflow
Priority	Enum(Low,Medium,High)	Public	Default=Medium
DueAt	DateTime	Public	Optional; if set, must be >= createdAt
GcalEventId	String	Public	Optional; immutable unless re-linked
createdAt	DateTime	Public	Auto-set
updatedAt	DateTime	Public	Auto-updated

Table: Operation description for Task class

Operation	Visibility	Return type	Argument	Pre-Condition	Post Condition
createSubtask	Public	Subtask	title:String	Caller can edit task	Subtask created, parent progress unaffected
changeStatus	Public	void	newStatus:Enum	Valid transition; not archived project	Status updated; activity logged; notifications queued

Operation	Visibility	Return type	Argument	Pre-Condition	Post Condition
assignTo	Public	void	userId:UUID	User in workspace; permissions ok	Assignee set; Slack notification queued
linkToCalendar	Public	void	calendarId:String	DueAt set; Google connected	Event created/updated; GcalEventId set

5) Subtask

Table: Subtask class

```
Subtask
+Id:UUID
+TaskId:UUID
+Title:String
+AssigneeId:UUID
+Status:Enum
+createdAt:DateTime
+updatedAt:DateTime
+toggle()
```

Table: Attributes description for Subtask class

Attribute	Type	Visibility	Invariant
Id	UUID	Public	Not NULL
TaskId	UUID	Public	Must reference existing Task
Title	String	Public	Not NULL, length 1..140
AssigneeId	UUID	Public	Optional; must be workspace member
Status	Enum(ToDo,Done)	Public	Default=ToDo
createdAt	DateTime	Public	Auto-set

Attribute	Type	Visibility	Invariant
updatedAt	DateTime	Public	Auto-updated

Table: Operation description for Subtask class

Operation	Visibility	Return type	Argument	Pre-Condition	Post Condition
toggle	Public	void	–	Caller can edit parent task	Status flips; parent completion % recalculated

6) Comment

Table: Comment class

```
Comment
+Id:UUID
+TaskId:UUID
+AuthorId:UUID
+Body:String
+createdAt:DateTime
+editedAt:DateTime
+addMention()
+edit()
+delete()
```

Table: Attributes description for Comment class

Attribute	Type	Visibility	Invariant
Id	UUID	Public	Not NULL
TaskId	UUID	Public	Must reference existing Task
AuthorId	UUID	Public	Must reference existing User
Body	String	Public	Not NULL; length 1..5000; sanitized markdown

Attribute	Type	Visibility	Invariant
createdAt	DateTime	Public	Auto-set
editedAt	DateTime	Public	Optional; ≥ createdAt

Table: Operation description for Comment class

Operation	Visibility	Return type	Argument	Pre-Condition	Post Condition
addMention	Public	void	mentionedId:UUID	Mentioned user in workspace	Notification queued; activity logged
edit	Public	void	newBody: String	Within edit window OR author has permission	Body updated; editedAt set
delete	Public	void	–	Author or admin	Comment removed or marked deleted (soft)

4.6 CalendarEventLink

Table: CalendarEventLink class

```
CalendarEventLink
+Id:UUID
+TaskId:UUID
+CalendarId:String
+EventId:String
+SyncedAt:DateTime
+createOrUpdate()
+unlink()
```

Table: Attributes description for CalendarEventLink class

Attribute	Type	Visibility	Invariant
Id	UUID	Public	Not NULL
TaskId	UUID	Public	Must reference existing Task
CalendarId	String	Public	Valid Google calendar id
EventId	String	Public	Not NULL after creation
SyncedAt	DateTime	Public	Auto-updated on sync

Table: Operation description for CalendarEventLink class

Operation	Visibility	Return type	Argument	Pre-Condition	Post Condition
createOrUpdate	Public	void	taskSnapshot:JSON	Task has DueAt; user connected to Google	Event inserted/patched; SyncedAt set
unlink	Public	void	–	Caller authorized	Link removed; event optionally deleted per policy

4.7 WorkspaceMember (join)

Table: WorkspaceMember class

```
WorkspaceMember
+WorkspaceId:UUID
+UserId:UUID
+Role:Enum
+AddedAt:DateTime
+AddedBy:UUID
+remove()
+changeRole()
```

Table: Attributes description for WorkspaceMember class

Attribute	Type	Visibility	Invariant
WorkspaceId	UUID	Public	FK → Workspace.Id
UserId	UUID	Public	FK → User.Id
Role	Enum(Owner,Admin,Manager,Member,Viewer)	Public	Not NULL; exactly one Owner across members **per workspace**
AddedAt	DateTime	Public	Auto-set
AddedBy	UUID	Public	FK → User.Id; must be Owner/Admin

Constraints: Primary key (WorkspaceId, UserId) unique; a user must exist; cascading delete when Workspace deleted; removing last Owner forbidden.

Table: Operation description for WorkspaceMember class

Operation	Visibility	Return type	Argument	Pre-Condition	Post Condition
remove	Public	void	userId:UUID workspace_id:UUID	Caller Owner/Admin; not last Owner	Row deleted; Workspace.updatedAt bumped; audit logged
changeRole	Public	void	userId:UUID, workspace_id:UUID, newRole:Enum	Caller Owner/Admin; role valid	Role updated; audit logged; enforce at least one Owner

4.8 ProjectMember (join)

Table: ProjectMember class

```
ProjectMember
+ProjectId:UUID
+UserId:UUID
+Role:Enum
+AddedAt:DateTime
+AddedBy:UUID
```

```
+remove()
+changeRole()
```

Table: Attributes description for ProjectMember class

Attribute	Type	Visibility	Invariant
ProjectId	UUID	Public	FK → Project.Id
UserId	UUID	Public	FK → User.Id; must already belong to Project's Workspace
Role	Enum(Manager,Contributor,Viewer)	Public	Not NULL
AddedAt	DateTime	Public	Auto-set
AddedBy	UUID	Public	FK → User.Id; Manager+

Constraints: Primary key (ProjectId, UserId) unique; cascading delete when Project deleted; visibility rules derive from membership if project is private.

Table: Operation description for ProjectMember class

Operation	Visibility	Return type	Argument	Pre-Condition	Post Condition
remove	Public	void	userId:UUID, projectId:UUID	Caller Manager+	Row deleted; Project.updatedAt bumped; audit logged
changeRole	Public	void	userId:UUID, projectId:UUID, newRole:Enum	Caller Manager+	Role updated; audit logged

4.9 TaskWatcher (join)

Table: TaskWatcher class

```
TaskWatcher
+TaskId:UUID
```

```
+UserId:UUID
+AddedAt:DateTime
+AddedBy:UUID
+remove()
```

Table: Attributes description for TaskWatcher class

Attribute	Type	Visibility	Invariant
TaskId	UUID	Public	FK → Task.Id
UserId	UUID	Public	FK → User.Id; must be in the same Workspace as Task
AddedAt	DateTime	Public	Auto-set
AddedBy	UUID	Public	FK → User.Id; editor of the task

Constraints: Primary key (TaskId, UserId) unique; cascade on Task delete; adding a watcher triggers a Notification preference check.

Table: Operation description for TaskWatcher class

Operation	Visibility	Return type	Argument	Pre-Condition	Post Condition
remove	Public	void	userId:UUID, taskID: UUID	Caller can edit task or is the user	Row deleted; audit logged